ZIX - Informe Técnico del Proyecto

Asistente personal de IA por voz para Windows (v0.2) - Estado real verificado del código
Fecha de generación: 07/09/2026 - Versión: 2.0 (ampliación: visión, roadmap, equipo + mapa extendido) - Rama main, commit 422db9a - Informe basado únicamente en inspección directa del repositorio, pruebas ejecutadas y logs de sesiones en vivo.

1. Resumen del proyecto 2
2. Arquitectura 2
3. Funcionalidades implementadas 2
4. Sistema de voz 3
5. Inteligencia artificial 3
6. Herramientas y dependencias 4
7. Configuración e instalación desde cero 4
8. Estructura del código 4
9. Pruebas y diagnóstico 4
10. Problemas actuales (priorizados, con evidencia) 5
11. Seguridad 5
12. Estado del proyecto 5
13. Próximos pasos 5
14. Conclusión 5
15. Visión y evolución futura de ZIX 6
16. Roadmap del proyecto 6
17. Metodología de trabajo en equipo 6
18. Contexto para OpenCode y ChatGPT 6
19. Mapa de estado actual y futuro 7
20. Decisiones de arquitectura (existentes, con motivo) 7
21. Reglas para futuros cambios 7
22. Historial de actualizaciones 7
Mapa real del proyecto 8

1. Resumen del proyecto

Qué es ZIX. Asistente personal de IA por voz para Windows, inspirado en JARVIS. Núcleo: **Micrófono -> Wake Word -> STT -> Agent -> Tools -> TTS -> Altavoz**, con CLI de texto como adaptador de pruebas.

Objetivo principal. Conversar por voz con el PC en español: activar por palabra clave propia ("Hey ZIX"), dictar órdenes, que el agente las ejecute con herramientas (abrir apps, calcular, hora) y responder hablando.

Problema que resuelve. Operar Windows por voz con componentes gratuitos/locales donde es posible (wake word y TTS 100% locales; STT y LLM vía tier gratuito de Groq).

Estado actual (verificado 05/09/2026). v0.2 funcional y validado en vivo con el modelo propio: ciclo completo voz operativo (wake 0.7457 > 0.5, STT 200, agente 200, TTS audible, tools ejecutadas). Suite: 60/60 tests verdes. Pendiente de cerrar: re-activación del wake tras expirar la ventana (en diagnóstico, causa sin determinar) y segundo commit con trabajo no registrado.

Figura 1. Pipeline de voz de ZIX v0.2.

2. Arquitectura

Capas: **1) Interfaces** (CLI texto *main.py*; pipeline voz *voice_main.py*) -> **2) Núcleo agnóstico de I/O** (*core/agent.py*: bucle razona-actúa-observa, máx. 10 iteraciones; *core/tool_registry.py* + *tool_executor.py*) -> **3) Proveedores intercambiables** (interfaces *ai_provider.py*, *stt_provider.py*, *tts_provider.py*, *wake_word_provider.py*; implementaciones Groq/Piper/openWakeWord) -> **4) Configuración** (*config.py* + *.env*). La máquina de estados (*states.py*: IDLE, LISTENING, THINKING, SPEAKING, CONVERSATION) vive en *conversation.py*. El Agent nunca importa groq directamente: solo *groq_provider.py* lo usa.

Comunicación: el loop de *voice_main.py* alimenta cada bloque de audio al detector y a la conversación; al cerrarse una frase, STT transcribe a texto, el Agent decide (llamadas a tools con esquemas JSON validados por ToolExecutor), Piper sintetiza y se reproduce. Todo síncrono y bloqueante (v0.2, un solo hilo).

3. Funcionalidades implementadas

Funcionalidad	Qué hace / cómo	Estado
Agente conversacional con tools	Bucle ReAct con historial de sesión, tool-calling JSON validado. Archivos: core/agent.py, tool_registry.py, tool_executor.py	Funcionando (verificado en vivo 05/09)
Tools: calculator, echo, open_app, time_tool	Calculadora por AST (sin eval), abrir apps por lista blanca (15), hora, eco. En vivo abrió Chrome real	Funcionando
CLI de texto	Adaptador de pruebas del Agent. main.py	Funcionando
Wake word propio zix.onnx	DNN openWakeWord entrenada en Colab (5000+2000+5000 clips). models/wakeword/zix.onnx 13802 B + .data 200704 B. Verificado 0.7457	Funcionando (ver 4.x)

Funcionalidad	Qué hace / cómo	Estado
Captura WASAPI nativa	48 kHz estéreo -> 16 kHz mono con resampler propio + selección por nombre. audio.py	Funcionando
STT Whisper Groq	español, modelo whisper-large-v3-turbo. Transcripciones correctas en vivo	Funcionando
TTS Piper local	Voz es_MX-ald-medium 63 MB, 100% offline. Respuestas audibles en vivo	Funcionando
Máquina de conversación	Ventana 10 s, VAD, validación de órdenes. conversation.py	Funcionando
Scripts setup/verify	setup_voice_models.py idempotente; verify_zix_onnx.py (4 pasos). integration_test_*.py manuales	Funcionando
UI Stitch exportada	apps/zix-ui/index.html (26 KB): orbe, estados, verificado en navegador sin errores	Parcial (maqueta, sin backend)

Leyenda de estado: Funcionando = probado en vivo o por tests; el único punto con problemas se detalla en la sección 4 (re-activación post-IDLE).

4. Sistema de voz

Wake Word. Modelo DNN propio *zix.onnx* (openWakeWord, inference onnx), umbral 0.5, bloques de 1280 muestras @16 kHz. Respaldo intacto: hey_jarvis. Verificación 05/09: 375 bloques, shapes [1,16,96]->[1,1], score 0.7457.

Micrófono. Microphone Array Realtek por WASAPI en formato nativo (48 kHz/2 ch), medido con señal viva (rms 0.148). Resampler propio en *audio.py* (fix pendiente de commit: *windows[::k]*).

STT / IA / TTS. Whisper-large-v3-turbo (es) vía Groq; agente qwen/qwen3.6-27b con fallback openai/gpt-oss-20b ante rate-limit; Piper es_MX-ald-medium local. Todo probado en vivo con HTTP 200.

Estados y flujo. IDLE (solo detector) -> LISTENING (VAD: rms>0.02, mín. 0.3 s habla, 0.8 s silencio) -> THINKING (STT+agente) -> SPEAKING (TTS) -> CONVERSATION (10 s sin wake) -> IDLE. El detector corre en TODOS los estados sin excepción.

Problema actual de audio/detección. Reportado por el usuario: tras expirar a IDLE, "Hey ZIX" no siempre reactiva. Auditoría de código (05/09): sin guards, locks, cooldowns ni contadores en *voice_main.py*, *conversation.py* ni *openwakeword_wake_word.py*; *reset()* existe pero nada lo llama (igual que los ejemplos oficiales). Medición en vivo: scores ~0.0002 con ambiente fuerte. Causa exacta SIN determinar: hipótesis principal, desajuste acústico (sala ruidosa/reverberante vs entrenamiento limpio); stream y máquina de estados verificados sanos. En diagnóstico, sin cambios aplicados.

5. Inteligencia artificial

Proveedores. Groq (nube, tier gratuito): chat *qwen/qwen3.6-27b* + fallback *openai/gpt-oss-20b* solo ante *RateLimitError*; STT *whisper-large-v3-turbo*. Locales: openWakeWord (wake), Piper (TTS), VAD propio.

Solicitudes. *groq_provider.py*: mensajes + herramientas al endpoint OpenAI-compatible, timeout 30 s; rate-limit/timeout/respuesta inválida se traducen a errores propios; *Agent.run* devuelve mensaje de error legible en vez de lanzar.

Contexto y memoria. Historial por sesión en memoria (*Agent._messages* con system prompt en español); *reset_session()* lo borra. Sin persistencia: no hay memoria entre ejecuciones (verificado en código).

Configuración. *config.py* (dataclass inmutable) + *.env* (ignorado por git). Claves necesarias (nombres, sin valores): GROQ_API_KEY, GROQ_MODEL, GROQ_FALLBACK_MODEL, MAX_AGENT_ITERATIONS, GROQ_TIMEOUT_SECONDS, LOG_LEVEL, VOICE_*, AUDIO_INPUT_*, WAKE_WORD_*, STT_*, PIPER_*, VOICE_RUNTIME_DIR (lista completa en *.env.example*, solo placeholders).

6. Herramientas y dependencias

Tecnología	Para qué
Python >=3.10 + .venv	Runtime y entorno aislado Windows
groq / python-dotenv / numpy / sounddevice	LLM+STT / config .env / audio y DSP
openwakeword + onnxruntime	Detección wake word local (inferencia onnx)
piper-tts + espeak-ng-data	Síntesis de voz offline + fonemas
pytest (60 tests)	Suite verde: agent, audio(21), conversación(18), VAD, registry, calculator
Google Colab T4 + Piper v2.0.0 + MIT RIRs	Entrenamiento del modelo propio (notebook train_hey_zix.ipynb)
Stitch (Google, vía MCP)	Diseño UI exportado a apps/zix-ui/
Graphify (427->439 nodos) + vault Obsidian (9 páginas)	Base de conocimiento del proyecto
GitHub (ZXcris/zix-bot-project, rama main)	Registro remoto, push al día hasta 422db9a

7. Configuración e instalación desde cero

Requisitos (determinados del proyecto): Windows 10/11 64-bit, Python 3.10+, micrófono y altavoces, internet (Groq + descargas iniciales), cuenta Groq gratuita para la API key. GPU no requerida en PC (solo se usó T4 en Colab para entrenar). Pasos: 1) clonar repo y crear .venv; 2) *pip install -e .* (+ pytest); 3) copiar *.env.example* a *.env* y poner GROQ_API_KEY; 4) *python scripts/setup_voice_models.py* (descarga Piper 63 MB + espeak a %LOCALAPPDATA%\zix por la limitación ASCII de espeak-ng); 5) el modelo *zix.onnx* ya viene integrado vía *WAKE_WORD_MODEL=zix*; 6) *python -m zix.voice_main*.

8. Estructura del código

Ruta	Responsabilidad
src/zix/main.py	CLI temporal de pruebas del Agent (v0.1)
src/zix/voice_main.py	Entrada de voz: construye pipeline y loop de bloques
src/zix/config.py	Config inmutable desde .env + defaults
src/zix/core/	agent.py (ReAct), ai_provider.py, groq_provider.py, tool*.py
src/zix/tools/	calculator, echo, open_app (whitelist), time_tool
src/zix/voice/	audio, conversation, states, vad, providers STT/TTS/wake
scripts/	setup_voice_models, integration_test_*, wakeword_training/
models/ (ignorado)	wakeword/zix.onnx+.data, piper/, espeak-ng-data/
tests/ (6 archivos)	60 pruebas: agent, audio, conversación, VAD, registry, calculator
apps/zix-ui/	Maqueta web exportada de Stitch (nuevo, sin backend)
docs/ + wiki Obsidian	SESSION_STATE.md, informe actual, vault ZIXVault (9 págs)

9. Pruebas y diagnóstico

Suite pytest (ejecutada 05/09): 60/60 verdes en ~1-3 s (agent 6, audio 21, conversación 18, VAD 6, registry 3, calculator 6). **verify_zix_onnx.py re-ejecutado 05/09** sobre el modelo integrado: 375 bloques, shapes OK, 0.7457 > 0.5. **Sesiones en vivo** (logs 05/09): 5+ ciclos wake->STT->agente->TTS con HTTP 200, apertura real de Chrome vía tool, respuestas audibles.

Fallos observados. 1) Rate limit Groq 429 puntual (28/08 y 05/09): el fallback lo absorbió y un reintento dio 200; causa: cuota free-tier. 2) No re-disparo post-IDLE reportado por el usuario: auditoría sin hallazgo de compuertas; scores en vivo ~0.0002 con sala ruidosa; causa exacta pendiente (hipótesis principal: desajuste acústico, no fallo de código). 3) Entrenamiento Colab: fallos resueltos

documentados (weights_only torch 2.13, torchcodec, RIR corruptos, keys YAML, onnxscript/onnx_tf en tflite -ignorable-).

10. Problemas actuales (priorizados, con evidencia)

Críticos. (1) Re-activación tras IDLE no fiable (reporte usuario 05/09; auditoría código limpio; scores vivos en suelo; causa SIN determinar - en diagnóstico).

Importantes. (2) Cuota gratuita Groq con 429 esporádicos (logs; mitigado con fallback+reintento). (3) Trabajo sin registrar: fix audio.py, logging voice_main, tests ampliados, apps/zix-ui, wiki suelta (git status). (4) Entrenamiento depende de VM efímera Colab (~1 h por recovery; pipeline documentado).

Menores. (5) LOG_LEVEL=DEBUG temporal activo en .env (pendiente revertir). (6) Mojibake de encoding en docs/wiki. (7) Ruido de line-endings en .gitignore. (8) Conversión tflite falla (onnx_tf ausente; irrelevante: el .onnx no la necesita).

11. Seguridad

API keys. GROQ_API_KEY solo en *.env* local, ignorado por git (verificado); *.env.example* solo placeholders; *setup_voice_models.py* declara no imprimir secretos; barrido de secretos en HEAD vacío. Riesgo estándar: clave en texto plano local (sin cifrado; aceptable single-user, no subirlo).

Ejecución. *open_app.py*: lista blanca de 15 apps, *Popen sin shell=True*, rechaza lo demás (verificado en código). *calculator.py*: AST restringido, sin *eval* crudo. Riesgo honesto: la whitelist incluye terminal/cmd y explorer: el modelo puede ABRIR consolas pero no ejecutar comandos en ellas.

Archivos/permisos. WAVs de utterances/responses con tu voz quedan en *runtime/* (ignorado por git, pero en claro en disco). Sin autenticación ni sandbox: app monousuario local por diseño, sin superficie de red entrante.

12. Estado del proyecto

ZIX v0.2 es un asistente de voz **real y operativo** en este PC: modelo propio verificado, ciclo de voz validado en vivo varias veces con herramientas ejecutadas de verdad, 60/60 tests y remoto sincronizado. No es maqueta ni plan: lo afirmado aquí se comprobó en código, pruebas y logs. Límites honestos: un fallo abierto (re-wake post-IDLE, causa sin determinar), rate limits del tier gratuito, y trabajo reciente aún sin commitear.

13. Próximos pasos

Correcciones necesarias. 1) Resolver re-wake post-IDLE con prueba controlada (scores por bloque ya instrumentados). 2) Revertir LOG_LEVEL a INFO. 3) Registrar segundo commit (audio fix + tests + UI) y push.

Mejoras recomendadas. 4) Umbral/ganancia calibrable del wake (datos, no intuición). 5) Saludo breve al activar (hoy solo aviso impreso). 6) Limpiar wiki suelta y PNGs o decidir su sitio.

Nuevas funcionalidades. 7) Conectar la UI Stitch (*apps/zix-ui/*) como frontend real. 8) Empaquetado .exe Windows. 9) Memoria entre sesiones (hoy solo sesión en RAM).

14. Conclusión

ZIX está al ~85%: núcleo, voz completa con modelo propio, herramientas y pruebas, todo verificado en vivo. Falta cerrar un bug real (re-activación), registrar el trabajo pendiente en git y decidir el camino de UI/empaquetado. Nada de lo esencial está sin construir; lo que falta es estabilizar, no inventar.

15. Visión y evolución futura de ZIX

Esta sección es VISIÓN FUTURA: no significa que estas funcionalidades estén implementadas (ver Mapa, sección 19, para el estado real de cada una).

La dirección acordada es convertir ZIX progresivamente en un asistente personal de IA para Windows capaz de: conversar por voz; recordar información útil; utilizar herramientas; interactuar con el PC de forma controlada; ejecutar acciones mediante herramientas seguras; tener una interfaz visual; manejar mejor errores y estados; ser ampliable con nuevos providers y tools; y eventualmente distribuirse como una aplicación sencilla para Windows.

Base real que ya sostiene esa visión (verificado en código): arquitectura por providers intercambiables (*ai_provider.py*, *stt/tts/wake_word_provider.py*), registro de tools con esquemas validados, máquina de estados de voz y maqueta UI en *apps/zix-ui/* aún sin backend.

16. Roadmap del proyecto

Fase 0 - Estabilización. Resolver el re-wake tras IDLE (en diagnóstico); confirmar fixes pendientes (*audio.py*, tests, UI); registrar trabajo en Git; revertir LOG_LEVEL=DEBUG temporal a INFO; suite verde; revalidar ciclo completo de voz.

Fase 1 - Preparación para trabajo en equipo. Cristian y su primo trabajarán juntos en el futuro. Dejar listo: *main* como rama estable; ramas individuales por persona/tarea; convención de nombres; commits atómicos con mensaje claro; Pull Requests con revisión; resolución de conflictos; no editar la misma parte sin coordinar; revisar Git (*status*, rama, commits, cambios) antes de cada tarea. NOTA: el primo AÚN NO se incorpora (sin PC ni cuenta GitHub): solo queda preparada la estructura (ver sección 17).

Fase 2 - Robustez. Wake resistente al ruido, calibración del detector (umbral/ganancia con datos), mejor manejo de errores, logging útil, recuperación ante fallos y más pruebas automatizadas.

Fase 3 - Memoria. Diseñar persistencia separando: sesión, persistente, preferencias, información importante y contexto temporal. REGLA PREVIA: definir qué se almacena y cómo se elimina antes de implementar.

Fase 4 - Expansión de tools. Nuevas herramientas Windows, cada una con propósito, entrada definida, validación, manejo de errores, tests y límites de seguridad. Sin acceso ilimitado al sistema.

Fase 5 - UI. Convertir la maqueta Stitch (*apps/zix-ui/index.html*, hoy estática y sin backend) en interfaz real conectada a ZIX.

Fase 6 - Distribución. Instalador/.exe Windows con instalación sencilla (hoy solo hay ejecución desde fuente con .venv).

17. Metodología de trabajo en equipo

Guía para Cristian y su primo. **Regla principal: primero entender el estado actual del proyecto; después modificarlo.** Antes de tocar código revisar: 1) *git status*, 2) rama actual, 3) últimos commits, 4) cambios sin commit, 5) documentación (*README*, *docs/*, vault), 6) trabajo reciente del otro, 7) archivos de la tarea. Dividir tareas por componente; una rama por tarea; commits pequeños con mensaje claro; sincronizar con *pull --rebase* antes de subir; Pull Request + revisión del otro antes de merge a *main*; conflictos: hablar primero, resolver juntos, nunca force-push a *main*; comunicar cambios importantes en el log del vault. Detalle operativo en la página [[Trabajo-en-equipo]] del vault.

18. Contexto para OpenCode y ChatGPT

Todo agente que trabaje en ZIX debe: leer *README.md*, *docs/SESSION_STATE.md*, *.env.example* y *graphify-out/GRAPH_REPORT.md* antes de modificar código; revisar el código real, Git y cambios recientes; no asumir funcionalidades (contrastar con la sección 19); no duplicar lo existente; no borrar código sin justificarlo; no sobrescribir trabajo ajeno; ejecutar tests tras cambios; informar qué modificó y

qué comprobó; señalar incertidumbres; y jamás exponer secretos. Este informe + el vault + el grafo son el contexto de entrada.

19. Mapa de estado actual y futuro

Estados: Implementado (probado) / Parcial / En diagnóstico / Planeado / Futuro. Ampliación del mapa de la sección 14 con próxima fase y prioridad.

Componente	Estado actual	Problemas	Próxima fase	Prioridad
Agente + tools	Implementado	Rate 429 esporádico (mitigado)	Fase 4 (más tools)	Media
Wake zix.onnx	Implementado*	Re-wake post-IDLE en diagnóstico	Fase 0 (resolver)	Alta
Mic WASAPI	Implementado	Sala ruidosa degrada scores	Fase 2 (calibración)	Alta
STT / Agent / TTS	Implementado	Cuota gratuita	Fase 2 (errores)	Media
Conversación	Implementado	-	Fase 2 (logging)	Media
CLI / scripts	Implementado	-	-	Baja
UI Stitch	Parcial	Maqueta sin backend	Fase 5	Media
Entrenamiento Colab	Parcial	VM efímera; tflite ignorable	-	Baja
Docs/wiki/grafo	Parcial	Wiki suelta; mojibake	Fase 1 (ordenar)	Media
Memoria persistente	Planeado	Sin diseño aún	Fase 3	Media
Distribución .exe	Futuro	Sin empaquetado	Fase 6	Baja
Trabajo en equipo	Planeado	Primo aún sin PC/cuenta	Fase 1	Media

20. Decisiones de arquitectura (existentes, con motivo)

Ninguna inventada: cada motivo sale de comentarios/docs del propio código. Lo futuro va marcado RECOMENDACIÓN.

1) **Providers desacoplados por interfaces.** *core/agent.py*: 'El Agent NO importa groq'. Motivo: cambiar a Ollama/otros sin reescribir. 2) **WASAPI nativo + resampler propio.** Formato nativo del dispositivo y conversión a 16 kHz/mono (*audio.py*, commit d12e33c). 3) **espeak-ng en %LOCALAPPDATA% ASCII.** El wheel de piper no lee rutas no-ASCII como 'Imágenes' (*piper_tts.py*). 4) **openWakeWord local + Groq gratuito.** Política de gratuito/local donde sea posible (README). 5) **Whitelist open_app sin shell.** Seguridad por diseño (*open_app.py*). 6) **Calculator por AST, sin eval** (*calculator.py*). 7) **VAD + ventana 10 s + orden válida.** Definición aprobada en *conversation.py*. 8) **Un solo hilo síncrono** (*conversation.py*: 'todo el flujo es síncrono (v0.2)'). 9) **Config inmutable + .env** (*config.py* frozen dataclass). 10) **Modelo propio entrenado en Colab** por límite de hardware local. 11) **.onnx.data junto al modelo** (onnxruntime external data). 12) **Layout src/ setuptools** (*pyproject.toml*). RECOMENDACIÓN: saludo al activar, umbral calibrable, empaquetado y memoria según roadmap.

21. Reglas para futuros cambios

No romper lo existente al agregar; providers desacoplados; tools validadas; tests siempre verdes; jamás secretos al repo; sin dependencias innecesarias; sin permisos ilimitados al agente; nada 'terminado' sin probarlo en vivo o por tests; documentar cambios importantes en el vault.

22. Historial de actualizaciones

Versión	Fecha	Contenido
1.0	05/09/2026	Informe inicial: 14 secciones + mapa real. Verificación completa del código.
2.0	07/09/2026	Ampliación: visión futura, roadmap Fases 0-6, metodología de equipo, contexto para agentes, mapa extendido, decisiones de arquitectura, reglas e historial.

Mapa real del proyecto

Componente	Estado	Evidencia	Problemas	Próximo paso
Agente + tools	Funcionando	En vivo: abrió Chrome real; 60 tests	Rate 429 esporádico (mitigado)	Nada urgente
Wake zix.onnx	Funcionando*	0.7457 en grabación; detecciones en vivo	*Re-wake post-IDLE en diagnóstico	Prueba controlada de scores
Mic WASAPI	Funcionando	Señal viva medida (rms 0.148)	Sala ruidosa degrada scores	Calibrar umbral/ ganancia
STT Whisper	Funcionando	Transcripciones correctas en vivo	Requiere internet+cuota	Nada urgente
Agent Groq+fallback	Funcionando	HTTP 200 en vivo	Cuota free-tier	Nada urgente
TTS Piper	Funcionando	Respuestas audibles en vivo	-	Nada urgente
Conversación/estados	Funcionando	Ventanas 10 s, expiración normal	-	Nada urgente
CLI texto	Funcionando	main.py operativo	-	Nada urgente
Scripts setup/verify	Funcionando	Idempotentes, re-ejecutados	-	Nada urgente
UI Stitch	Parcial	Maqueta verificada sin errores	Sin backend	Definir frontend real
Entrenamiento Colab	Parcial	Modelo generado y descargado	VM efímera; tflite falla (ignorable)	Nada (modelo listo)
Docs/wiki/graph	Parcial	Vault 9 págs; grafo 439 nodos	Wiki suelta sin sitio; mojibake	Ordenar y commitear